Go2 SDK Development Guide

Search article titles

About Go2
Remote Controller
SDK Concepts 3
Application Development 7
Software Interface Services 19
Motion Services Interface V2.0
DDS Services Interface
Avoidance Services Interface
VuiClient Interface
RobotStateClient Interface
Motion Switcher Service Interface
Sports Services Interface
AI motion Services interface
Basic Services Interface
LiDAR Services Interface
Multimedia Services Interface
UWB Services Interface
Fault Services Interface
SLAM and Navigation Services Interfa
Aruco Recharge Service Interface
Odometer Services Interface
D1 Mechanical Arm Services Interface
ROS2 Services Interface
Python Services Interface
Case reference 8
Debugging Specification
FAQ

DDS Services Interface

Last Updated On：2025-09-24 18:25:17

This page contains content

Introduction to Unitree Go...
Interface Description
unitree::robot::ChannelFactory
unitree::robot::ChannelPublis...
unitree::robot::ChannelSubscr...
Go2 Service Client Interfa...
List of some general error co...
Partial Client Common Interfa...
Other service component inte...

Introduction to Unitree Go2 Sdk

unitree_sdk2 It is a layer of encapsulation on DDS that supports QoS configuration of DDS components, provides a simple encapsulation interface for application development, and implements an RPC mechanism based on DDS Topic. It is suitable for data communication in different scenarios through publish/subscribe and request/response methods between internal and external processes in Go2.

> **Explanation:**
> **unitree_sdk2** The installation and configuration tutorial will not be repeated in this article. Please refer to the "**Quick Start**" section

Interface Description

unitree::robot::ChannelFactory

`unitree::robot::ChannelFactory` provides a single instance of ChannelFactory under Unitree:: robot to create a DDS Topic based Channel. ChannelFactory must call the initialization interface before use to initialize the underlying DDS configuration. The calling method is as follows
`unitree::robot::ChannelFactory::Instance()->Init(0)`

The descriptions of each interface are as follows:

Function Name	Instance
Function Prototype	static unitree::robot::ChannelFactory* unitree::robot::ChannelFactory::Instance()
Functional Overview	Obtain a singleton static pointer
Parameters	None
Return Value	Unitree::robot::ChannelFactory singleton static pointer
Remarks	

Function Name	Init
Function Prototype	void Init(int32_t domainId, const std::string& networkInterface = "", bool enableSharedMemory = false)
Functional Overview	Initialize ChannelFactory by specifying three initialization parameters: Domain Id, network card name, and whether to use shared memory.
Parameters	**domainId**: Default domain id for constructing DdsParticipant； **networkInterface**: Specify the network card name, default to blank; **enableSharedMemory**: Whether to use shared memory, default to false.
Return Value	None
Remarks	If the **networkInterface** is empty, automatic selection of network card configuration will be generated; When developing applications outside of Go2, enableSharedMemory needs to be set to false.

Function Name	Init
Function Prototype	void Init(const std::string& ddsParameterFileName = "")
Functional Overview	Initialize DdsParticipant by specifying a configuration file.
Parameters	**ddsParameterFileName**: Specify a configuration file path in JSON format. When ddsParameterFileName is empty, The default is the configuration file named dds_parameter.json in the current program directory.
Return Value	None
Remarks	If the file does not exist, an exception will be thrown indicating that the configuration file does not exist; If the specified network is not available, a DDS exception will be thrown.

Function Name	Init
Function Prototype	void Init(const JsonMap& param)
Functional Overview	Specify JsonMap configuration to initialize DdsParticipant.
Parameters	**param** JsonMap format configuration.
Return Value	None
Remarks	If the specified network is not available, a DDS exception will be thrown.

Function Name	CreateSendChannel
Function Prototype	template ChannelPtr CreateSendChannel(const std::string& name)
Functional Overview	Specify name to create a Channel with a specified message type of MSG for sending data.
Parameters	**name**：Channel Name.
Return Value	template ChannelPtr
Remarks	

Function Name	CreateRecvChannel
Function Prototype	template ChannelPtr CreateRecvChannel(const std::string& name, std::function<void(const void

Helpful
Feedback

	*)> callback, int32_t queuelen = 0)
Functional Overview	Specify name to create a Channel with a specified message type of MSG for receiving data.
Parameters	**name**：Channel Name； **callback**: callback function when the message arrives; **queuelen**: The length of the message cache queue; If the length is 0, do not enable columns in t he message cache.
Return Value	template ChannelPtr
Remarks	If the message takes a long time to process in the callback function, it is recommended to enabl e the message cache queue to avoid blocking the DDS callback thread.

Function Name	**Release**
Function Prototype	void Release()
Functional Overview	Release ChannelFactory static resources.
Parameters	None
Return Value	None
Remarks	unitree::robot::ChannelFactory::Instance()->Release()

unitree::robot::ChannelPublisher

`unitree::robot::ChannelPublisher` Class implements the message publishing function of the specified type

Class name	**Constructors and Destructors**
template ChannelPublisher	explicit ChannelPublisher(const std::string& channelName); ~ChannelPublisher()

Explanation:

unitree::robot::ChannelPublisher Used during construction unitree::robot::ChannelFactory::Instance()->CreateSendChannel(mChannelName)，So when using unitree::robot::ChannelPublisher Before that, you need to initialize the unitree::robot::ChannelFactory singleton.

Function Name	**InitChannel**
Function Prototype	void InitChannel()
Functional Overview	Initialize the Channel and prepare it for sending messages.
Parameters	None
Return Value	None
Remarks	

Function Name	**CloseChannel**
Function Prototype	void CloseChannel()
Functional Overview	Close Channel
Parameters	None
Return Value	None
Remarks	

Function Name	**Write**
Function Prototype	bool Write(const MSG& msg)
Functional Overview	Send a specified type of message.
Parameters	msg: Send a message of type MSG.
Return Value	true: Successfully sent; false: Sending failed.
Remarks	

unitree::robot::ChannelSubscriber

`unitree::robot::ChannelSubscriber` Implemented the specified type of message subscription function.

Class Name	**Constructors and Destructors**
template ChannelSubscriber	explicit ChannelSubscriber(const std::string& channelName); ~ChannelSubscriber()

Attention:

unitree::robot::ChannelSubscriber Used during construction unitree::robot::ChannelFactory::Instance()->CreateSendChannel(mChannelName)，So when using unitree::robot::ChannelSubscriber initialization is required before proceeding unitree::robot::ChannelFactory singleton。

Function Name	**InitChannel**
Function Prototype	void InitChannel(const std::function<void(const void*)>& callback, int64_t queuelen = 0)
Functional Overview	Initialize the Channel and prepare it for receiving or processing messages.
Parameters	**callback**: callback function when the message arrives; **queuelen**: The length of the message cache queue; If the length is 0, do not enable columns in t he message cache.
Return Value	None
Remarks	If the message takes a long time to process in the callback function, it is recommended to enabl e the message cache queue to avoid blocking the DDS callback thread.

Function Name	**CloseChannel**
Function Prototype	void CloseChannel()
Functional Overview	Close Channel。
Parameters	None
Return Value	None
Remarks	

Function Name	GetLastDataAvailableTime
Function Prototype	int64_t GetLastDataAvailableTime()
Functional Overview	Gets the last time a message was received.
Parameters	None
Return Value	Returns -1 when the Channel is not initialized; Otherwise, it returns a timestamp that monotonic ally increments from 0 starting from system startup, accurate to microseconds.
Remarks	

Go2 Service Client Interface

Multiple service components within Go2 provide partial functional interfaces through RPC Client, allowing users to easily access Con trol or data acquisition of Go2.

List of some general error codes

Number	Error description	Remarks
3001	Unknown error	Client/server returned
3102	Request sending error	Client returned
3103	API not registered	Client returned
3104	Request timeout	Client returned
3105	Request and response data do not m atch	Client returned
3106	Invalid response data	Client returned
3107	Invalid lease	Client returned
3201	Response sending error	Occurred on the server and will not be returned to the client
3202	Internal server error	Server returned
3203	API not implemented on the server	Server returned
3204	API parameter error	Server returned
3205	Request rejected	Server returned
3206	Invalid lease	Server returned
3207	Lease already exists	Server returned

Partial Client Common Interface Description

Function Name	Init
Function Prototype	void Init()
Functional Overview	Client initialization, completing logic such as client API registration.
Parameters	None
Return Value	None
Remarks	

Function Name	SetTimeout
Function Prototype	void SetTimeout(float timeout)
Functional Overview	Set the RPC request timeout time.
Parameters	timeout: in seconds.
Return Value	None
Remarks	If no timeout is set, the default timeout time is 1 second.

Function Name	WaitLeaseApplied
Function Prototype	void WaitLeaseApplied()
Functional Overview	Applying for a lease, the function will block until the lease is applied for.
Parameters	None
Return Value	None
Remarks	Only valid when lease is enabled.

Function Name	GetApiVersion
Function Prototype	const std::string& GetApiVersion()
Functional Overview	Obtain the client API version.
Parameters	None
Return Value	Returns the client API version.
Remarks	

Function Name	GetApiServerVersion
Function Prototype	const std::string& GetServerApiVersion()
Functional Overview	Obtain the server API version.
Parameters	None
Return Value	Returns the server API version.
Remarks	

Other service component interfaces

Multiple service components within Go2 provide partial functional interfaces to external users through RPC clients, such as the **SportClient** class, **RobotStateClient** class, **VuiClient** class, **ObstaclesAvoidClient** class.

For specific interface descriptions, please refer to "Obstacle Avoidance Service Interface", Volume and Light Service Interface", "Device Status Service Interface", "High level Sports Service Interface"Documentation.

The PDF has been downloaded. Rate us: